



Keyspark

Real-time detection of input automation
over a Kafka + Spark streaming pipeline

Murat Emirhan Aykut

Bahçeşehir University

BDA5011 Big Data Analytics - June 2026



One unbounded input stream, one question



The data

Every key press, mouse move, click and scroll. An unbounded, high-rate, bursty event stream - data that arrives, not data that sits.



The application

Jigglers, auto-clickers and keep-awake tools fake presence. Flag, per minute, whether input came from a human or a tool.



Why it is a Big Data problem

Event-time windowing, out-of-order arrival, producer vs consumer rate mismatch, fault tolerance, and a fast-vs-complete split.



Privacy by construction

Only key identifiers and timings are kept. No raw typed text is ever persisted - the whole signal is content-free.

The pipeline is the contribution. The detector is the application that gives the stream a purpose.

Streaming systems meet behavioral signals



Streaming foundations

Kafka: the replayable commit log
[Kreps 2011](#)

Spark RDDs: fault-tolerant engine
[Zaharia 2012](#)

Structured Streaming: stream as a growing table
[Armbrust 2018](#)

Lambda architecture: speed + batch
[Marz 2015](#)



Streams of interaction data

Kafka pipeline over a browser clickstream
[Pal 2023](#)

Same Kafka + Spark Lambda stack, other domain
[Demirezen 2023](#)

TimeAware: per-time-bin self-monitoring
[Kim 2016](#)



Keystroke + mouse dynamics

Keystroke timing as a behavioral biometric
[Monrose 2000](#)

Emotional state from keystroke timing
[Epp 2011](#)

Mental fatigue from keystroke dynamics
[Acien 2022](#)



Detecting automation

Passive game-bot detection from input
[Gianvecchio 2009](#)

Human vs bot vs cyborg accounts
[Chu 2012](#)

BeCAPTCHA-Mouse: synthetic bot trajectories
[Acien 2022](#)

KeySpark's niche: a live streaming pipeline computing a content-free human-vs-automation signal over combined keyboard + mouse events.

A self-collected, content-free event archive

$\sim 2.65 \times 10^6$

events, 25 active days, one user, one machine
frozen 18 Jun 2026



Key identifiers + timings only. No raw text is ever stored.



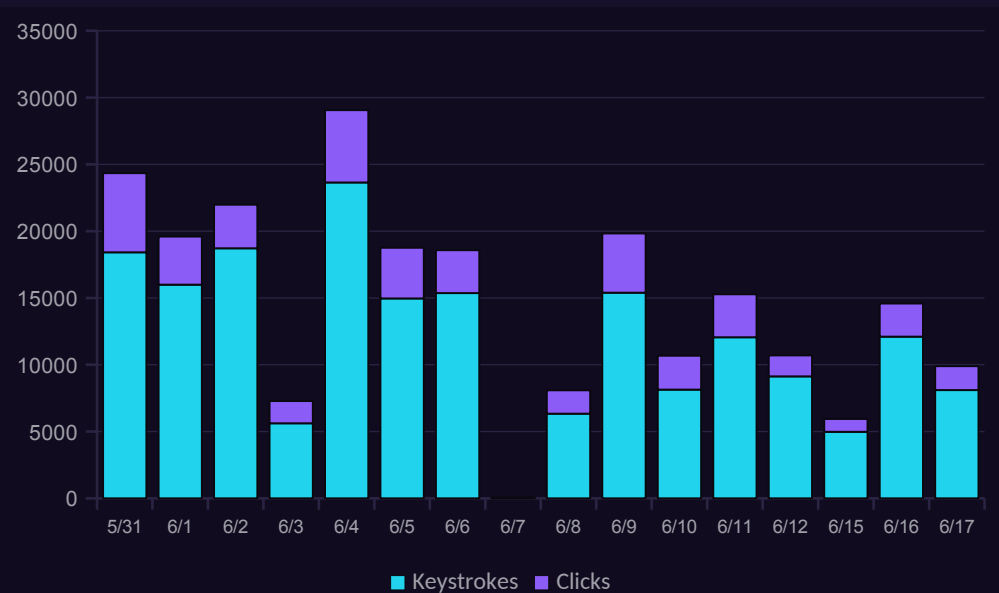
Mouse moves throttled to ~50/s. Clicks and scrolls never throttled.



JSON events parsed with an explicit schema, stored as Parquet.

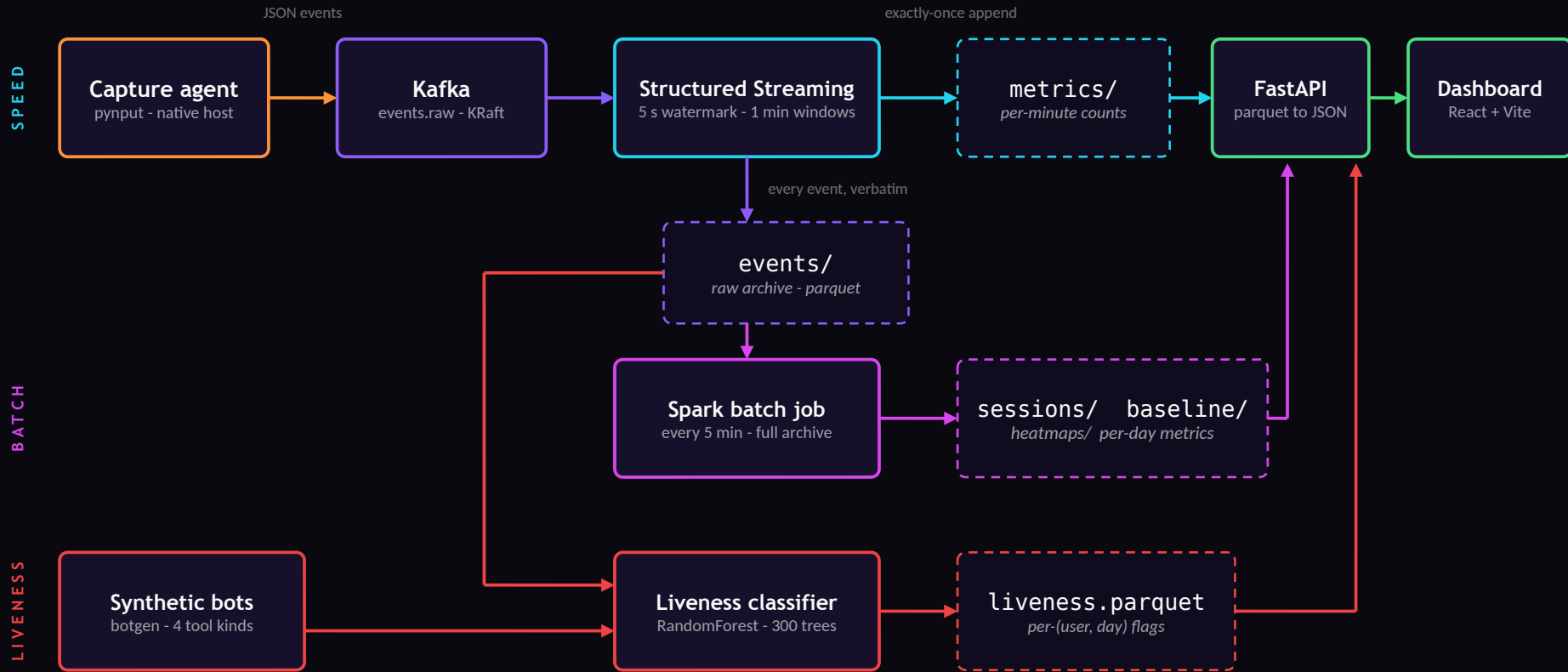
```
type key x y button pressed dx dy user ts
```

Daily captured activity (from output/metrics, May 31 - Jun 17)



Real archive data. Words and corrections are counted from the same keystrokes; ~50/s throttled mouse moves are archived but not shown.

One stream in, three kinds of truth out



Why this stack - every piece earns its place



Apache Kafka

ingestion log

A durable, replayable, ordered log decouples the agent from Spark: capture never blocks, bursts are buffered, and offsets are what make replay and exactly-once possible. A direct DB write or a fire-and-forget queue gives none of this.



Structured Streaming

speed layer

Event-time windows plus a watermark stay correct under late, out-of-order arrival - a naive consumer loop cannot. Checkpointed offsets and atomic Parquet commits give exactly-once. The same code runs on local[*] and on a cluster.



Spark batch

batch layer - Lambda

Sessions and inter-event timing depend on the previous event; lag() cannot see across micro-batches. So the stream lands everything, and a batch pass over the bounded archive is the source of truth. That split is the Lambda architecture, by name.



Parquet

serving store

Columnar, compressed, splittable - shaped for the dashboard's analytical scans. It is the file sink Spark commits to exactly-once, and it adds zero services to operate.

Why not just write events to MongoDB?



What MongoDB is built for

- Point reads and writes of mutable documents
- Secondary indexes for selective lookups
- Flexible per-document schema
- Genuinely simpler to stand up for low-rate CRUD



What this workload is

- Append-only, high-rate event stream - nothing is ever updated
- Windowed aggregation over everything, not point lookups - columnar scans win
- Needs event-time windows, watermarks, exactly-once - Mongo has none; we would rebuild them in app code
- Needs replay for recovery and reprocessing - the Kafka offset log gives it for free

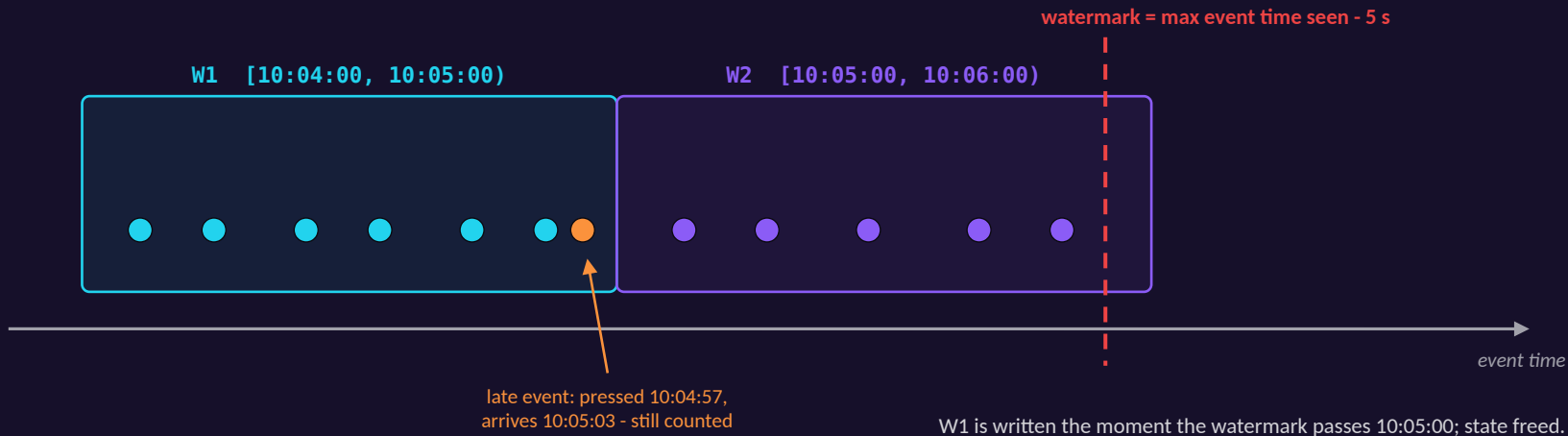


Not a bad database - the wrong shape. On Mongo we would hand-build windowing, late-event handling, exactly-once and replay in application code. Kafka + Spark + Parquet give exactly those, natively. The honest flip side: for low-rate mutable CRUD, Mongo would be the simpler and better choice.

Event-time windows and the 5 s watermark

event time when the key was pressed (ts in the event) - windows are defined on this

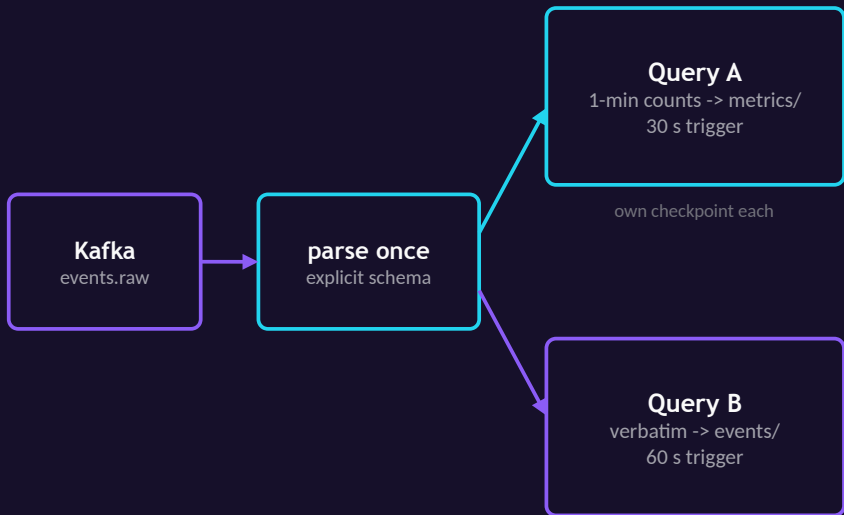
processing time when Spark happens to see it - irrelevant to the result



Why the watermark is mandatory: the Parquet sink is append-only - each window's row is emitted exactly once, never rewritten. Without a watermark Spark could never declare a window closed, so append could emit nothing.

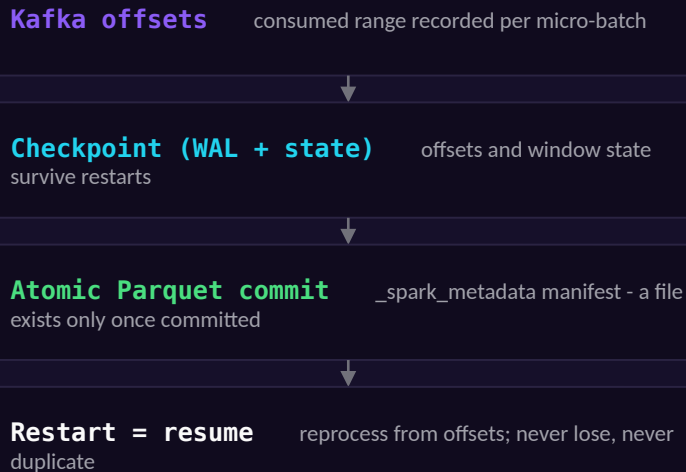
One stream, two queries, exactly-once on disk

One parsed stream, two sinks



Without triggers, every micro-batch wrote a part file (100k+ tiny files once).

Exactly-once, end to end



Sleep/wake hardening: failOnDataLoss=false resets to the earliest offset instead of crash-looping; restart loops + a watchdog bounce only the wedged component.

Liveness: was this minute typed by a human?

7 cross-modal shape features (per 1-min window)

key_diversity distinct keys / keystrokes

ks_iei_cv inter-keystroke interval CV

move_iei_cv inter-mouse-move interval CV

iei_cv all-event cadence CV

step_mean / step_cv mouse step - rigid for a juggler

mouse_fraction near 1 juggler, near 0 key tool

Shape, not volume: regularity and variety, never content. Windows with < 5 events abstain.



RandomForest - 300 trees, balanced classes

Human class = the real archive. Non-human class = synthetic events from botgen: a fixed-pattern juggler, an F15-style keep-awake, an auto-typer, an auto-clicker - cadence jittered so only the robotic shape remains. One featurizer for both classes: no train/serve skew.



A day turns red when ≥ 2 windows score ≥ 0.8

a sustained burst of automation, not one odd minute - surfaced on the dashboard calendar after each 5-min batch refresh

Honest caveat: the non-human class is synthetic. Validation on real captured automation is named future work.

Throughput first, the classifier second

DATA FROZEN - 18 JUN 2026

Pipeline performance - single machine, local[*]

Layer	Throughput	Latency
Spark batch (full pipeline)	$\sim 7.0 \times 10^5$ ev/s	-
Structured Streaming	$\sim 2.0 \times 10^5$ rows/s	275 ms / micro-batch
End-to-end (keypress to UI)	-	~ 65 s

~ 65 s = 60 s window + 5 s watermark. A per-minute metric cannot exist before its minute ends - the floor is the design, not the engine. Measured from Spark's own StreamingQueryProgress.

Liveness - stratified hold-out

Accuracy	0.994
Precision	0.970
Recall	0.994
F1	0.982
ROC-AUC	1.000

Top features: *iei_cv*, *step_cv*, *move_iei_cv* - the regularity tells.
1008-window hold-out.



Read the right-hand numbers carefully: the non-human class is synthetic, so the hold-out partly measures separation from our own generator. We claim the features are discriminative - we do not claim a field detection rate. Validation on real captured automation is future work.

The live demo, right after these slides



1 - Type

I type; within ~65 s the per-minute metrics, time series and heatmap update on the dashboard.

watch the window + watermark latency happen



2 - Inject a jiggle

botgen pushes synthetic jiggle events into Kafka; a sustained burst scores non-human and the calendar day flips red.

the whole pipeline reacts end to end



3 - Open the Spark UI

the Structured Streaming tab shows live input rate vs processing rate and micro-batch durations.

the engine's own evidence, not mine

Then a walk through the code: the streaming job, the batch job, and the featurizer.

What this is not, yet



Honest limits

- Single broker, single user, single machine (local[*]) - partitioning and shuffle never really stressed
- Synthetic non-human class - hold-out scores are inflated relative to automation in the wild
- Idle suppressors (caffeinate, Amphetamine, PowerToys Awake) emit no input events - invisible by construction
- ~65 s latency floor is tied to the 1-min window; shorter windows = noisier labels
- A sleep that kills Spark's local RPC costs a cold restart, not seconds



Future work

- Validate on real captured automation: a hardware USB jiggle emits genuine HID events the agent can capture (software injection is dropped by macOS)
- Ground botgen against the full spectrum of real keep-active tools
- Out-of-stream probe of OS power assertions to catch idle suppressors
- Multi-user ingestion to exercise Kafka partitioning; a real cluster to exercise Spark shuffle

Conclusion



Standard big-data parts, end to end: pynput -> Kafka -> Spark -> Parquet -> FastAPI -> React.



The Lambda split is earned, not decorative: cross-batch ordering forces a batch source of truth.



Streaming correctness is explicit: event time, a 5 s watermark, append + checkpoints = exactly-once.

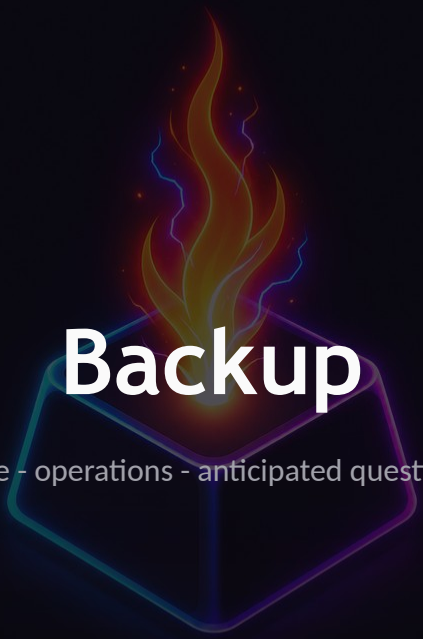


The ML rides the pipeline and is reported honestly: content-free features, synthetic-class caveat named.

Thank you.

Questions welcome - live demo next.



A glowing, multi-colored cube with a flame rising from its top center. The cube's edges are outlined in a vibrant blue and purple light. From the top face of the cube, a stylized flame rises, composed of orange and yellow wavy lines with small sparks. The entire graphic is set against a dark blue background.

Backup

code - operations - anticipated questions

keyspark/streaming_job.py - parse, window, sink

```
parsed = (  
    raw.selectExpr("CAST(value AS STRING) AS json")  
    .select(from_json(col("json"), event_schema()).alias("e"))  
    .select("e.*")  
    .withColumn("event_time", col("ts").cast("timestamp"))  
    .withWatermark("event_time", WATERMARK) # 5 seconds  
)  
  
metrics = (  
    parsed.groupBy(window(col("event_time"),  
                           WINDOW_DURATION), # 1 minute  
                  col("user"))  
    .agg(*event_count_exprs()) # keystrokes, words,  
                                   # corrections, clicks  
)  
  
metrics_query = (  
    metrics.writeStream.format("parquet")  
    .option("path", METRICS_PATH)  
    .option("checkpointLocation", METRICS_CHECKPOINT)  
    .outputMode("append")  
    .trigger(processingTime="30 seconds")  
    .start()  
)
```

explicit schema

no inference - predictable, cheap, typed

event-time watermark

bounds lateness; makes append legal on a windowed aggregate

tumbling window

per-(user, minute) groups on event time

checkpoint + append + trigger

exactly-once output; trigger caps file creation (30 s metrics, 60 s archive)

Checkpoints, sleep/wake, batch, serving



Checkpoint + commit anatomy

Per query: Kafka offset ranges (WAL) + window state. The Parquet sink's `_spark_metadata` manifest commits files atomically - readers only see committed files, so restart resumes without loss or duplicates.



Sleep/wake hardening

`failOnDataLoss=false` resets truncated offsets to earliest instead of crash-looping. Every process runs in a restart loop; a watchdog listens for the OS wake notification, polls output freshness, probes the Spark RPC, and bounces only the wedged pane.



Batch internals

Every 5 min from an in-process scheduler in the API. Reads the full archive by file glob - independent of the streaming commit log. Sessions via gap-and-island: order by event time per user, flag gaps > 5 min, cumulative-sum the flags into session ids.



Serving

FastAPI reads Parquet and serves JSON; React polls. The calendar joins per-day metrics with liveness flags; a flagged day renders red and drills down to that day's timeline and heatmap.

Likely probes, prepared answers

Why not Flink or Kafka Streams?

Lambda needs a batch engine anyway - Spark is one engine and one API for both layers. Flink's lower per-event latency would not help: the floor here is the 1-minute window itself.

Why a 5 s watermark?

One local broker produces little disorder, and the watermark delays emission: at 30 s the first result took ~90 s. 5 s covers observed lateness and keeps the dashboard live.

Why 1-minute tumbling windows?

The product unit is a minute of activity. Shorter = lower latency but noisier liveness labels; longer = smoother but staler.

Why a RandomForest, not a deep model?

7 tabular features and modest data. It retrains in seconds each batch cycle, resists overfitting with balanced weights, and its feature importances (the CV features dominate) explain its decisions.

How exactly is exactly-once achieved?

Offsets + window state checkpointed per micro-batch; the file sink commits atomically via `_spark_metadata`. Replay after failure reprocesses the same offsets; the sink never double-commits.

Could the agent write to Spark directly?

Then capture blocks on every consumer hiccup and nothing is replayable. The log decouples rates, buffers bursts, and is what makes restart and reprocessing free.